Fine-grained Device Management for Handling Simultaneous Read-Write Operations in an Interactive Media Proxy server

Raju Rangaswami [*] **Zoran Dimitrijevic** [†] **Edward Y. Chang** [‡]
Shueng-Han Gary Chan [§]

Abstract

Interactive media has recently gained a lot of interest. Interactivity gives viewers VCR controls like slow-motion, pause, fast-forward, and instant replay. Traditional server-based or client-based approaches for supporting interactivity either consume too much network bandwidth or require large client buffering; and hence they are economically unattractive. In this paper, we propose the architecture and design of IMP, an interactive media proxy server that transforms non-interactive broadcast or multicast streams into interactive ones for servicing a large number of end users. For IMP to work cost effectively, it must carefully manage its storage devices, which are needed for caching voluminous media data. In this regard, we propose a fine-grained device management strategy consisting of three complementary components: *disk profiler*, *data placement*, and *IO scheduler*. Through quantitative analysis and experiments, we show that these fine-grained strategies considerably improve device throughput under various workload scenarios.

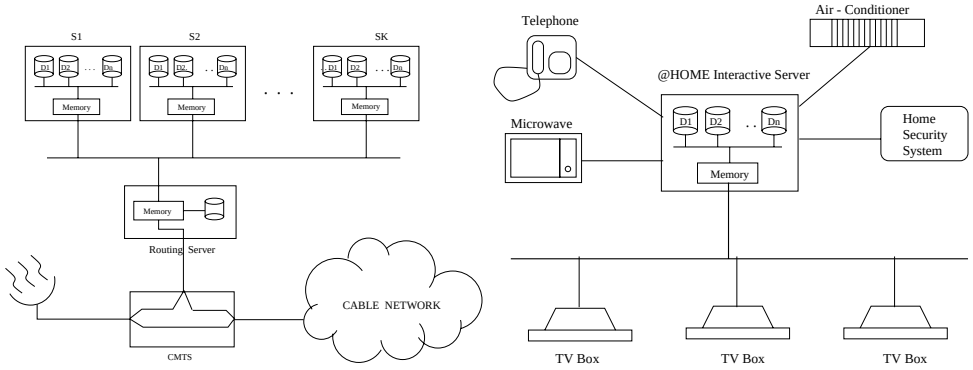
1 Introduction

Due to the proliferation of digital cameras and video content, managing video data effectively for supporting efficient retrieval has become increasingly important. For instance, on-demand interactive video streaming for education or entertainment over the Internet or broadband networks is gaining increasing interest [3, 11, 12, 24, 25]. Such streaming is often referred to as video-on-demand (VOD). In true-VOD, a video server allocates a dedicated stream to each user so that the user can freely interact with the video by means of VCR controls (such as pause, fast-forward, and instant replay). But such a system becomes expensive in both network and server bandwidth when tens of thousands of concurrent users need to be accommodated. A more scalable solution is to serve multiple requests for the same video with a single broadcast or a few multicast streams [1, 2, 9].

[*]Department of Computer Science, University of California, Santa Barbara. Phone: 1-805-893-7788. Email: raju@cs.ucsb.edu
[†]Department of Computer Science University of California, Santa Barbara Email: zoran@cs.ucsb.edu
[‡]Electrical & Computer Engineering, University of California, Santa Barbara. Email: echang@ece.ucsb.edu
[§]Department of Computer Science, Hong Kong University of Science and Technology. Email: gchan@cs.ust.hk

Because of the nature of broadcast and multicast, however, end users cannot interact with a TV program or a video using VCR-like controls. We propose IMP, an interactive media proxy server architecture which acts as a dual client/server system to enable interactivity. As a client of broadcasters (or multicasters), the IMP system reduces network traffic to support interactivity. Additionally, it acts as a server by managing streams to enable interactivity for a large number of end users. With interactive capability, students in a virtual classroom or at a library can watch a live lecture at their own pace. A hotel can turn televised programs or movies into interactive ones in its guest rooms. A cable service provider can enable interactivity for thousands of subscribers. We believe that the IMP architecture is attractive because it not only solves the scalability problem of the traditional server-based VOD model, but also provides a cost-effective solution to user interactivity.

The IMP architecture faces challenging design issues. In addition to the high bandwidth and real-time IO requirements that have been studied extensively [3, 15, 20, 21, 22, 27], it must deal with writes as well as reads requests, and the changing ratio of write requests to read requests. (We define the ratio of write requests to read requests as the *write-read ratio*.) Let us explore this parameter further with three application scenarios:



(a) Local Area System (b) @HOME System

Figure 1: Interactive System Architectures.

1. *Local-area interactive system* : A local-area interactive system (Figure 1(a)) consists of a centralized bank of disk-array servers that provide service to a large number of clients in a local area. The local-area interactive system receives broadcast (or multicast) signals and enables interactivity for its clients. This model is clearly more economical than the server-based VOD model in which each viewer, interactive or not, requires some system resources and a dedicated wide-area network channel. In addition, on the client side, no caching device is required. In the local-area interactive service architecture, there are large numbers of interactive users and relatively few broadcast/multicast channels. This means that the design in this case should optimize for reads rather than for writes.

2. *@HOME interactive system* : The @HOME interactive system (Figure 1(b)) provides service simultaneously to multiple display devices in a classroom, at a library, or at home. The server has large data storage capacity in the form of a disk array and a moderate amount of memory. The system records broadcast programs on disk and serves interactive streams to multiple display devices. The @HOME service can be configured to provide interaction capability for certain popular channels in order to reduce cost. At the same time, it serves interactive content to the few users at home. Thus, in the @HOME interactive system, the number of reads and writes are approximately equal and optimization for both is important.

3. *Video surveillance system* : *Video surveillance systems* are used in a wide variety of settings — shopping malls, casinos, correction centers, banks etc. These systems can be made more effective if the surveillance team is provided with the ability to time-shift the surveillance streams. Typically, surveillance cameras are situated in numerous locations spanning the facility but are monitored by only a few people at a time. Thus, it receives much more recording (write) requests than interactive read requests.

In this paper, we propose the architecture and design of the IMP system. The IMP system employs various fine-grained device management strategies to address the traditional requirements of high-bandwidth IO and real-time operation, as well as changing write-read request ratio. Specifically, our fine-grained approach can be characterized by three key contributions:

Data Organization : First, we propose a high-level organization scheme for MPEG data. This scheme enables the IMP system to adapt to different write-read ratios and thus cater to a wide range of application scenarios. We show that an effective data organization scheme must consider at least three design parameters: *disk latency*, *data transfer rate*, and *IO resolution*. We define *IO resolution* as the ratio of the useful data read to the total data read from the hard disk. Since the improvement on one performance factor might lead to the degradation of the others, trade-offs between these design parameters must be carefully considered.

Device Profiling : Disk abstractions (e.g., SCSI or IDE interface) hide low-level device characteristics from the operating system and virtualize the access to the device in the form of logical blocks. This device abstraction makes the task of tuning disk operation to match application requirements (and thus improve IO efficiency) very difficult. Using Scsibench [7], our disk profiling tool, we collect detailed parameters directly from a disk instead of using vendor-provided data. In fact, we observe that even two disks of the same model from the same vendor may exhibit different performance characteristics, and some data (such as the locations of bad sectors) simply cannot be obtained ahead of time.

Device Management : We use the information obtained from *device profiling* to assist making accurate and effective decisions regarding data placement and IO scheduling. Intelligent decisions thus made can improve effective disk utilization, ensure that a sequential placement is indeed sequential, and keep response time short, thus improving the overall disk access efficiency of the IMP system.

Parameter	*Description*
$\mathcal{N}_w$	*Number of broadcast channels (write streams)*
$\mathcal{N}_r$	*Number of interactive requests (read streams)*
ρ	*Write-read Ratio*
f	*Fraction of Interactive Streams (i.e., fast-scans)*
$\mathcal{TR}$	*Minimum disk data transfer rate*
$\gamma(d)$	*Worst-case latency function to seek d cylinders*
$\mathcal{H}$	*Head-switch time*
$\mathcal{DR}$	*Average display rate of MPEG stream*
$\mathcal{DR}_f$	*Average display rate of a fast-scan stream*
C	*Throughput of the disk (number of requests served)*
$\mathcal{T}$	*IO cycle time*
$\mathcal{T}_s$	*Reservation time-slot*
$\mathcal{M}$	*Available system memory*
Z_l	*Logical zone set*
α , β	*Adaptive Tree parameters*
κ	*Number of exclusive I-frame sub-streams*
h	*Height of the Adaptive Tree*
η	*Density of the Adaptive Tree*
Φ	*IO resolution*

Table 1: System Parameters.

Our analytical study and experimental results suggest that such a fine-grained strategy can indeed be effective, and can offer performance improvement of as much as 75% over traditional systems. The rest of the paper is organized as follows. In section 2, we describe disk features obtained from our feature extraction tool. Section 3 describes how to place video streams intelligently on disk. Section 4 proposes *step-sweep*, an efficient IO scheduling algorithm to access stream data. In Section 6, we present a case-study using three sample configurations of our IMP system and evaluate our fine-grained device manager. In Section 7, we present related research and finally offer our conclusion in Section 8.

2 SCSI Disk Profiler

In this section, we present a SCSI disk feature extraction tool [7] that extracts disk parameters for the design of our data placement and IO scheduling policies. We first discuss the disk parameters that are of interest. We then explain how we extract these parameters. In Table 1, we enumerate these parameters along with other parameters that are introduced later.

Before we present disk features of interest, we provide a brief overview of the disk architecture (Figure **??**). The main components of a modern day disk drive are: one or more *disk platters* or recording surfaces rotating on a shared *spindle*, a set of *read-write heads* residing on a shared *disk arm*, *disk logic*, *cache memory* with embedded caching and replacement logic, and the *IO bus*. Data is stored on the hard disk by logically organizing it into disk *blocks* (the unit of disk access). Typically, a block corresponds to a disk *sector*. The set of sectors on the

same magnetic surface and that are at the same distance from the central spindle form a *track*. The set of tracks at the same distance from the spindle form a *cylinder*. To perform a data transfer, the disk arm must first move from its current position to the destination cylinder. It must then wait for the desired sector to rotate and reach the disk head before data transfer can commence. These constitute the disk seek and rotational delays respectively.

2.1 Disk Features

In addition to disk seek times and rotational delay, we are interested in the following disk parameters:

- Per-zone data transfer rate: On a modern disk, the amount of data that can be stored on a track is proportional to the length of the track. Outer tracks can store more data and hence have higher data transfer rates. A modern disk is partitioned into zones, each comprising multiple cylinders. The zones have different numbers of sectors per track and hence different data transfer rates. We extract per-zone data transfer rates to perform bit-rate matching for video streams (Section 3).
- Spare/bad sector locations: A disk allocates information about spare sectors for relocating data when a good sector is damaged. A file that was created as a sequential one can be subsequently broken into non-sequential blocks by sector re-mapping. We collect spare/bad sector locations to ensure that a sequential placement is indeed sequential.
- Head switch and cylinder switch times: While accessing a block residing on the next track or the next cylinder, the disk arm needs to be re-positioned. The head switch and cylinder switch times must be accurately accounted for as a part of IO cost.

Traditional device management schemes often neglect some of the above disk parameters. Most studies make the worst-case assumption on disk parameters, and some employ statistical models to estimate them. These traditional schemes can be inefficient or ineffective. For instance, some schemes can be inefficient because they do not take advantage of the higher transfer rates of outer disk zones or do not effectively reduce rotational delay. Some are ineffective because they are not aware that their sequential placement could have been broken into non-sequential files due to sector relocation. To maintain peak disk performance, one must collect fine-grained information to make intelligent data placement and IO scheduling decisions. We discuss in Sections 3 and 4 how we use disk parameters to enhance device throughput.

2.2 Feature Extraction Methods

Our tool runs on a Linux system in the user mode, using SCSI generic interface to the SCSI bus. The SCSI library was implemented from scratch, complying to SCSI-2 standard [10]. We then developed methods to extract a range of disk features that are of interest. We used some of the

strategies presented in a similar study from HP Labs [26, 19]. All the timing measurements are performed using an Intel Pentium time stamp counter register, which provides high precision with the overhead of only one register-read instruction.

There are two methods for obtaining a disk parameter: an interrogative method and an empirical method. The interrogative method uses SCSI commands to query the disk. The empirical method performs actual measurements in real-time. The interrogative method may not be accurate since it returns the meta data provided by the vendor, and that data can be misleading or out-dated. The empirical method can obtain accurate data, but it takes much more time to complete.

We collect disk parameters in several steps. The first step is to measure the rotational time of the disk. This step can be done by either the interrogative or the empirical method, since the results are the same. The second step is to obtain detailed logical-to-physical (cylinder, track, sector) block mapping. Our tool uses both interrogative and empirical methods to extract accurate mappings. The output of this operation is the zoning information, together with a list of all tracks that contain a large number of spare or bad sectors.

Zone	*Cylinders*	T_{size}	$\mathcal{TR}$	$\mathcal{TR}_{max}$	$\gamma(1)$	$\mathcal{H}$
1	0-847	254	18.56	21.77	1104	883
2	848-1644	245	18.02	21.00	1108	885
3	1645-2393	238	17.49	20.40	1108	876
4	2394-3097	227	16.70	19.46	1115	890
5	3098-3758	217	15.99	18.60	1115	890
6	3759-4380	209	15.70	17.92	1105	884
7	4381-4965	201	14.83	17.23	1099	875
8	4966-5515	189	13.98	16.20	1124	901
9	5516-6031	181	13.39	15.51	1124	903
10	6032-6517	174	12.89	14.92	1109	886
11	6518-6960	167	12.38	14.32	1119	887

Table 2: Disk Zone Features.

The zoning information of the disk (Seagate ST39102LW) we tested is presented in Table 2. T_{size} denotes the number of tracks in a given zone, and $\mathcal{TR}$ denotes the transfer rate in MBps. The disk has 12 tracks per cylinder. $\mathcal{TR}_{max}$ denotes the maximum transfer rate in MBps of each zone calculated using rotational time and track size for that zone. Note that the difference between the transfer rates of outer and inner zones can be substantial. Also, the transfer rate for long sequential reads is less than the maximum throughput when exactly one track is read owing to the presence of substantial track and cylinder skews. While conducting our experiments, we also learned that zoning characteristics can be slightly different even between physical devices of the same model. This is because the actual mappings differ depending on the location of errors on magnetic surfaces detected during the low-level format. Of course, we cannot predict bad sector locations for a disk. For this reason, we believe that a device manager should extract

precise zoning information, and it should periodically validate the logical-to-physical mappings in order to guarantee peak disk performance[1].

Our tool also measures head switch and cylinder switch times. Table 2 lists estimated maximum values for head switch time ($\mathcal{H}$) and cylinder switch time $\gamma(1)$ (in microseconds).

Finally, we collect information on disk latency, including seek time and rotational delay. Due to space limitation, we do not present these here. Interested readers may refer to the extended version [17] for details. The parameters collected by the disk profiler can assist data placement and IO scheduling, which we discuss next.

3 Data Placement

Based on our experiments with disk feature extraction in the previous section, we note that disk features, if known accurately, can be used to improve device throughput. More specifically, our aim is to maximize throughput by minimizing both intra-stream and inter-stream seek and rotational delay, and by improving the effective data transfer rate.

In this section, we first describe the *adaptive tree scheme*, a high-level data organization scheme for improving IO resolution (i.e., effective disk utilization) and reducing intra-stream disk latency. This organization scheme forms the basis of our low-level disk placement policy. Next, we present our low-level disk placement of stream data, which reduces inter-stream disk latency and improves the effective data transfer rate.

3.1 High-level Data Organization

Without loss of generality, we can assume that an MPEG stream[2] consists of m frame-sequences; each sequence has δ frames on average, led by an I frame and followed by a number of P and B frames.

To support a K-times speed-up fast-scan, the system displays one out of every K frames. Allowing K to be any positive integer, however, can result in the system suffering from high IO, network bandwidth, memory and CPU overhead. This is because a frame that is to be displayed (e.g., a B frame) may depend on some frames that are to be skipped (e.g., an I and a P frame). The client/server dual system ends up having to read, stage (in main memory), and transmit to a client (e.g., a TV box) many more frames than the client displays. We intend to remedy this poor IO resolution problem with an intelligent high-level data organization scheme.

[1] If a sector becomes bad and its data is relocated to a spare sector, a sequential file is no longer physically contiguous on the disk. The device manager may not be able to relocate the entire file to maintain its physical contiguity, but it can be aware of the changed conditions and reserve more time for performing IO.

[2] The Advanced Television System Committee (ATSC) has adopted MPEG2 as the encoding standard of DTV and HDTV.

To avoid processing the frames that are to be skipped, we do not include any B frames in a fast-scan, and the playback system displays a P frame only if the P frame's dependent I frame is also included in the fast-scan. This restriction will not allow a user to request a fast-scan of any speed-up. Since VCR or DVD players support only three to five fast-scan speeds, we support only a few, say five, selected speeds of fast-scans.

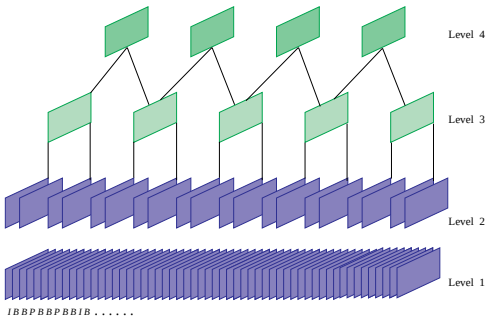


Figure 2: The Truncated Binary Tree Formation.

In the adaptive tree approach for high-level MPEG data organization, we use a basic truncated binary tree structure to store MPEG frames. To provide good IO resolution for all playback speeds, we organize MPEG frames in a truncated binary tree structure, as shown in Figure 2. The levels of the adaptive tree can be described as follows:

- *Level 1 (the leaf level):* The original sequential MPEG stream.
- *Level 2 :* All I and P frames stored in their playback sequence.
- *Level 3 to* $(\kappa + 2)$ *:* Containing only sampled I-frames. κ is the number of sub-streams containing only I-frames. The higher the level, the lower the sampling rate.

Each level of the tree forms a sub-stream of the original video stream and is stored as a sequential file on the disk. Where the different levels are placed on the disk will be explained as part of low-level disk placement (Section 3.2). To service a request, only one level of the tree is read from the disk, and hence high IO resolution can be achieved.

Let S_i denote the set of frames that belong to the ith level. An example of a five-level organization can be

$$S_1 = \{I_1, B_1, B_2, P_1, B_3, B_4, P_2, B_5, B_6, I_2 \cdots\}$$

$$S_2 = \{I_1, P_1, P_2, I_2, P_3, P_4, I_3 \cdots\}$$

$$S_3 = \{I_1, I_2, I_3, I_4 \cdots\}$$

$$S_4 = \{I_3, I_6, I_9, I_{12} \cdots\}$$

$$S_5 = \{I_6, I_{12}, I_{18}, I_{24} \cdots\}$$

When a nine-times fast-scan is requested, we can read in S_3 to achieve perfect resolution. (We assume that an I-frame leads a nine-frame sequence.) When a 54-times fast-scan is re-

quested, we read in S_5 to achieve perfect IO resolution.

Essentially, this adaptive tree approach trades storage space for IO efficiency. The precise trade-off can be controlled by fine-tuning the following two parameters:

- *Height (h).* The height parameter describes the number of levels (or files) in which the data is organized. The height parameter controls the number of fast-scan streams supported. Height is statically chosen and can be expressed as: $h = \kappa + 2$.
- *Density (η).* Density is a tunable system parameter whose value ranges from zero to one. Having a smaller η value, some levels of the tree can be eliminated to obtain lesser density. For example, at $\eta = 1/3$, only one out of every three levels of the tree (from the leaf level up) are kept. The higher the density, the higher the IO resolution (favorable) and storage cost (not favorable).

Another factor that affects the disk bandwidth requirement of a fast-scan request is the playback frame rate. A typical DVD player plays a fast-scan stream at 3 to 8 fps (frames per second), instead of 24 to 30 fps, the regular playback speed. A fast-scan stream needs to be displayed at a lower rate so that the viewer can comprehend and react in time to the content. As a consequence of having high IO resolution and lesser frame rate, a fast-scan stream under our adaptive tree scheme typically requires lower disk bandwidth than that of a regular playback stream.

3.2 Low-level Disk Placement

Now, we consider how we physically place each stream S_i on disk. Here, our goals are to maximize the effective data transfer rate and minimize disk latency. Our low-level data placement scheme achieves these goals by employing two independent solutions. We utilize *zoning* information to achieve higher transfer rates, and we perform intelligent *cylinder placement* in order to minimize disk latency.

3.2.1 Zoning

In the previous section, we described the adaptive tree scheme for MPEG data organization, which splits a stream into high and low bandwidth sub-streams. Higher bandwidth streams require higher data transfer rates to reduce overall data transfer time. From our experiments in disk feature extraction in Section 2, we realize that inter-zone bandwidth variations can be as great as 50% (see Table 2). Hence, there is a need to perform bit-rate matching of streams to zones.

In order to map streams to disk zones, we first create a set of logical disk zones, Z_l, from the physical zones of the disk. Let these logical zones be numbered from $1 \cdots |Z_l|$, from the outermost zone to the innermost. We map the physical zones of the disk to one of $|Z_l| (= \lceil h \cdot \eta \rceil)$

logical zones, where the η and h are the density and height parameters of the adaptive tree. Each sub-stream is mapped to one of these logical zones in which it is stored.

First, we assume that all streams (and sub-streams) have equal popularity to be accessed. (We will relax this assumption shortly.) Using adaptive tree levels, we divide each video stream into $|Z_l|$ sub-streams of differing bit-rates as described earlier. We group the similar bit-rate sub-streams from different video streams and place them in the same stream group S_i to obtain $S_1 \cdots S_{|Z_l|}$ such stream groups. Then the objective function to place each stream group S_i in a unique logical zone Z_j, can be written as:

$$\min \sum_{i=1}^{|Z_l|} \frac{\mathcal{DR}_i}{\mathcal{TR}_j}, \tag{1}$$

where $\mathcal{DR}_i$ denotes the common display rate of all sub-streams grouped in set S_i, and $\mathcal{TR}_j$ denotes the transfer rate of zone Z_j in which the sub-stream group S_i is placed. But, since not all streams enjoy the same popularity at all times, *is this the best objective function for optimal placement?*

If we can predict the future with high accuracy, we can possibly rewrite the objective function as:

$$\min \sum_{i=1}^{|Z_l|} \frac{\mathcal{DR}_i}{\mathcal{TR}_j} P(i), \tag{2}$$

where $P(i)$ denotes the ith sub-stream set's access probability. Unfortunately, predicting $P(i)$ is a lost cause. For instance, a sports highlight enjoys only a burst of interest.

However, if we look at the problem from a different perspective, zoning placement can provide a bound for the worst-case IO cycle time. A lower worst-case bound is useful because it conserves memory space for staging the streams.

[Lemma 3.1] *(Zoning Placement)*

Using $i = j$ in objective function (1) to place streams achieves the lowest worst-case bound for the total IO time for servicing any $\mathcal{N}$ requests.

While we leave the formal proof to the extended version [17], we use the following example to illustrate the idea.

[Example 3.2] *(Zoning Placement)*

Suppose an MPEG file is organized into three sub-streams, and their bit-rates are:

- S_1 (regular speed stream): 6 Mbps.
- S_2 (10 times fast-scan stream): 4 Mbps.
- S_3 (30 times fast-scan stream): 2 Mbps.

(As discussed in the previous section, fast-scan streams organized by the adaptive tree have lower bit-rates due to their high IO resolution and slower frame rates.) Suppose a disk have

three zones (Z_1, Z_2 and Z_3), and their transfer rates are 150, 125, and 100 Mbps, respectively. Suppose each zone can store only one stream, and the system services three requests in one IO cycle. Placing S_1 in Z_1, S_2 in Z_2, and S_3 in Z_3 gives the system the lowest worst-case total IO cycle time. The worst case total IO cycle time is $3 \times (6/150) = 12$ ms. It is easy to see that if we place S_1 in either Z_2 or Z_3, we have a higher worst-case total cycle time.

Although we introduce zoning in the context of storing MPEG video data, we can see that the above principles can be applied to general purpose multi-media file servers that support multiple data types such as images, audio, and video as well as traditional non real-time data. For instance, a text file can be stored in a low bandwidth disk zone, a 256 Kbps *mp3* file can be stored in a medium bandwidth zone, and a 19.2 Mbps HDTV stream can be stored in the high bandwidth zone to maximize disk throughput.

3.2.2 Cylinder placement

Intelligent stream placement on disk tracks can minimize seek and rotational overheads. Our system is characterized by N_w broadcast streams which need to be stored on disk and N_r interactive user streams. Using zoning, we combine similar bit-rate streams in the same compatible zone. We now describe how to place data for different streams which share a zone.

We note that write streams are deterministic in terms of start time, duration and average data-rate. However, because read streams depend on user interactions, they are inherently non-deterministic and unpredictable. The key idea of the cylinder placement strategy is to exploit the deterministic nature of write streams and use a best-effort approach for reads. To this end, we try to minimize the access overhead for write streams by grouping them in a span of a few adjacent cylinders.

To allocate disk cylinders to individual write streams, we follow a simple heuristic. For each stream, we allocate a group of adjacent cylinders of size c on the disk. A typical value for c could be 20 cylinders. Each consecutive write stream is allocated the next c cylinders on disk adjacent to the c allocated cylinders for the previous stream. When a set of cylinders $c \cdot \mathcal{N}_w$ is filled up with MPEG data, the entire set of write streams is moved to a different set of $c \cdot \mathcal{N}_w$ free cylinders within the same zone and adjacent to the previous cylinder set. We maintain the same relative cylinder distance between the stream pairs so that the scheduling order can be preserved over subsequent IO cycles. This minimizes *IO variability*. Minimizing IO variability is crucial for minimizing memory requirements [3]. Cylinder placement might lead to some fragmentation of disk space. However, we observe that for high bandwidth applications, the disk is bandwidth bound rather than storage bound. Hence some storage can be sacrificed for the sake of improving disk throughput.

Placing write streams in adjacent cylinder groups has the following advantages:

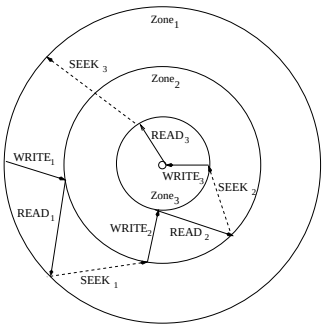


Figure 3: Step-sweep IO Scheduling.

1. The seek overhead for switching from one write stream to the next write stream requires the disk to seek c cylinders, typically a number less than 50. From our experiments to calculate disk seek time, we note that this overhead is almost equal to the minimum seek time for a single cylinder.

2. The log-structured file-system [18], proposes a similar strategy for optimizing writes by simply writing to a log instead of to individual files. However, our strategy of reserving cylinder groups for individual streams greatly reduces the probability that a single read operation may be non-sequential. Thus it offers much better read performance than LFS.

4 IO Scheduling

For a system to perform optimally under a given disk placement scheme, a closely coupled scheduling algorithm should be in place. We now propose our *step-sweep* IO scheduling algorithm which is based on our cylinder placement scheme.

Procedure: *Step-sweep*

- **Variables:**

  i : Logical zone variable

  Z_l : Set of logical disk zones

- **Execution:**

  1. Initialize $i = 1$
  2. Serve write requests in zone Z_i by sweeping in the direction toward the center of the disk.
  3. Serve read requests in zone Z_i. These read requests are served served in a predetermined order to reduce IO variability.
  4. If $(i == |Z_l|)$ then set $i = 1$
     Else set $i = i + 1$
  5. Go to step 2

Figure 4: Step-sweep scheduling algorithm

4.1 Step-Sweep

Given a constant amount of memory and disk bandwidth, step-sweep is designed to:

1. Maximize throughput,
2. Minimize response time.

The step-sweep algorithm operates as shown in figure 3. The disk-arm serves IO requests zone by zone, starting from the outermost zone. In each zone, it first serves the write requests by sweeping in the direction toward the center of the disk. In each zone, read requests are then scheduled in a pre-determined order which does not vary from one IO cycle to the next. The largest seek overhead in the case of read requests is restricted to the size (in tracks) of a single zone. The disk arm then moves to the adjacent inner zone and repeats this sequence. When the disk arm has served all the requests in the innermost zone, it gets re-positioned to the outermost zone of the disk. This completes one cycle of IO requests. This cycle is repeated over and over again. A formal algorithm for the step-sweep scheduling policy is presented in figure 4.

The design of step-sweep is based on the following considerations. First, we know that the write streams are deterministic in nature. The system has control on where it can place the write streams on disk. Using *Cylinder Placement*, the seek overheads for write streams can be minimized without suffering from IO variability at the expense of some fragmentation of disk space (see section 3). Thus, step-sweep schedules write streams optimally. However, read streams are unpredictable. Using simple sweep scheduling for read streams might result in large IO variability. This is due to the fact that the stream data is non-contiguous on disk. Hence, the service order for read streams in a zone must be pre-determined to minimize IO variability. Moreover, step-sweep reduces the seek overhead for read streams to the size of a single zone by serving one zone at a time.

In summary, *step-sweep* achieves its design objectives in the following manner:

1. To maximize throughput, we need to minimize *IO cycle time* as well as the memory use per stream. We define *IO Cycle Time* as the time required to complete a single round of IO for each stream served by the system. Reducing IO cycle time allows the system to service more users in a given period of time. Step-sweep reduces IO cycle time by minimizing seek overhead. When seek overhead is reduced, the system needs to pre-fetch less data to fulfill the real-time playback requirement for each stream, and this reduces data transfer time. Step-sweep minimizes memory use per stream by minimizing IO variability.

2. To minimize response time, we use a reservation time-slot, T_s, within each IO cycle time T. We use this reservation slot to schedule "unexpected" new requests with minimum delay. These unexpected requests might also include requests for non real-time data. Note that each new request can be served as soon as the current non pre-emptable IO operation is finished.

Thus the reservation time slot T_s is not necessarily contiguous, but can be used up in smaller chunks throughout the IO time cycle.

5 Quantitative Analysis

In this section, we present results obtained from our quantitative analysis[3]. Our quantitative model is developed assuming cylinder placement and step-sweep IO scheduling.

Let the IO cycle time be denoted by $\mathcal{T}$. This is the time required to complete a single round of IO for each stream served by the system. This IO cycle is repeated over and over again in the system. Let the disk support $\mathcal{N}_w$ broadcast channels, which perform simultaneous writes, and $\mathcal{N}_r$ time-shift streams, which perform reads. Suppose a fraction f of the time-shift streams are fast-scans and the ratio of write to read requests is ρ. Let C be the maximum throughput of the disk. Also, let $\mathcal{T}_s$ denote the worst case reservation slot time

If $\mathcal{T}_{Z_i}$ is the total time required to service all requests in zone i, and $\mathcal{T}_s$ is the reservation time slot, we can express $\mathcal{T}$, the cycle time, as:

$$\mathcal{T} = \sum_{i=1}^{p} \mathcal{T}_{Z_i} + \mathcal{T}_s \tag{3}$$

where, $p = |Z_l|$ is the number of logical zones used. In each zone, the scheduler first schedules the write requests, then the read requests. The disk arm is then moved to the beginning of the next zone by seeking to it. If the worst-case zone seek requires $\mathcal{T}_{zone}$ time, then:

$$\mathcal{T}_{Z_i} = \mathcal{T}_{W_i} + \mathcal{T}_{R_i} + \mathcal{T}_{zone} \tag{4}$$

Now, we proceed to quantify the write and read times in each logical zone. In each logical zone, Z_i, the disk performs $\mathcal{N}_w$ write requests. Thus, the time required to complete write operations in zone Z_i is:

$$\mathcal{T}_{W_i} = \mathcal{N}_w \cdot \Big[\gamma(d_w) + \frac{\mathcal{T} \times \mathcal{DR}(x)}{\mathcal{TR}_i}\Big] \tag{5}$$

where d_w is the average seek distance for write operations, $\mathcal{TR}_i$ is the average transfer rate of logical zone i. $\mathcal{DR}(x) = \mathcal{DR}$ for writing regular playback streams and $\mathcal{DR}(x) = \mathcal{DR}_f$ for fast-scan streams. The total read time in the entire cycle is given by:

$$\sum_{i=1}^{p} \mathcal{T}_{R_i} = \mathcal{N}_r \cdot \Big[\gamma(d_r) + \mathcal{T} \cdot (\frac{(1-f) \cdot \mathcal{DR}}{\mathcal{TR}_1} + \frac{f \cdot \mathcal{DR}_f}{avg_{i=2}^{p}(\mathcal{TR}_i)})\Big] \tag{6}$$

Substituting equations 4, 5 and 6 in equation 3, we can obtain a closed form solution for cycle time $\mathcal{T}$.

[3]Please refer to the extended version of this paper [17] for detailed quantitative analysis.

To fulfill real-time data requirements, each stream, read or write, allocates two buffers. The size of one buffer must be large enough to sustain the playback before another buffer is replenished. (Since this double-buffer scheme has been employed by many systems, we do not discuss it further. Please refer to [15] for details.) The memory requirement for a fast-scan stream is given by $2 \cdot \mathcal{T} \cdot \mathcal{DR}_f$ whereas that for write streams and regular-speed read streams is given by $2 \cdot \mathcal{T} \cdot \mathcal{DR}$. The total memory usage cannot exceed the available memory $\mathcal{M}$. We can thus quantify the memory requirement as:

$$\mathcal{M} \geq 2 \cdot \mathcal{T} \cdot \mathcal{DR} \times \left[\mathcal{N}_w + (1-f) \cdot \mathcal{N}_r + f \cdot \mathcal{N}_r \cdot \frac{\mathcal{DR}_f}{\mathcal{DR}}\right] \tag{7}$$

Given $\mathcal{M}$, $\mathcal{DR}$, $\mathcal{DR}_f$, $\mathcal{TR}$, $\gamma(d)$, f, Φ and the write-read ratio, ρ, we can use Equations 3 and 7 to estimate the throughput of the disk.

6 System Evaluation

In this section, we perform evaluate the performance of the IMP system and present results. In order to evaluate the IMP system, we must first define the performance metric. Given a system with fixed resources, we would like to maximize the number of video streams that can be supported simultaneously. Hence we measure the system performance in terms of the number of streams that can be supported by the disk. We refer to this performance metric as *disk throughput*. In particular, we perform the following experiments to evaluate the disk throughput.

Effect of System Configuration: We perform a case study on three sample configurations of the *adaptive tree* data organization scheme (Section 3). We study the effect of the following three system parameters on the throughput ($\mathcal{N}$):

- Available System Memory ($\mathcal{M}$),
- Write-Read Ratio (ρ), and
- Fraction of Interactive Streams (f).

Keeping two of the above parameters fixed, we examine the effect of changing the third.

Effect of Data Management Strategies: We examine the individual as well cumulative effect of the following fine-grained device management strategies on the throughput ($\mathcal{N}$):

- Zoning placement,
- Cylinder placement, and
- Step-sweep IO scheduling.

For a realistic study, we use actual disk trace support provided by our Scsibench tool (see Section 2). Scsibench supports the execution of disk traces with primitive disk commands

like seek, write, and read. It also provides very accurate timing tools of the order of a CPU clock cycle. Using Scsibench, we performed trace execution on the Seagate ST39102LW disk, presented in Table 2. To circumvent the physical limitations of present-day hard-drives, we then perform detailed analytical evaluation of the IMP system.

6.1 Effect of System Configuration

In this section, we perform a case study on three sample configurations of the *adaptive tree* data organization scheme (Section 3). The three tree configurations are designed to perform optimally under disjoint load conditions. We evaluate the performance of each configuration under varying system configuration.

First, we present three examples of tuned tree configurations, each of which is designed to optimize on a subset of the design parameters so as to perform optimally for a specific class of target applications, introduced in Section 1.

1. *Sequential or SEQ ($\eta = \frac{1}{h}$)* In the sequential configuration, only one out of h levels is stored to obtain a tree with density $\frac{1}{h}$. The sequential configuration stores only Level 1 of the tree. Higher levels of the tree are simply not stored. The frames are stored in the same order in which they are received. The goal of this scheme is to reduce seek overhead for writes thus conserving memory use in write-intensive applications. However, this scheme suffers from very poor IO resolution for fast-scans. This scheme would be practical for a *video surveillance* type application.

2. *Truncated Binary Tree or TBT ($\eta = 1$)* This is the normal configuration in which all levels of the tree are stored. In the truncated binary tree (TBT) approach, we attempt to provide good IO resolution at all fast-scan speeds and to maintain low disk latency for disk reads by trading some disk storage and write seek overhead. The advantage of this scheme is that we have prepared streams for supporting each fast-scan speed and hence can achieve 100% IO resolution and minimum disk latency at the same time. However, due to replication of data, storage cost increases. A *local-area interactive service* supporting a large number of clients could use this configuration.

3. *Partial TBT or PTBT ($\eta = 0.5$)* In the PTBT configuration, the tree has less density. With $\eta = 0.5$, we store only alternate levels of the TBT tree. Thus, in this configuration, there are prepared streams only for some of the fast-scan speeds. A fast-scan stream that is not directly accessible is created by selectively sampling the frames in another fast-scan stream, which serves a lower speed. Such streams suffer from a degraded IO resolution. The number of files to be written into (i.e., the number of seeks by the disk-arm) for each stream is $\lceil \eta \cdot h \rceil$. This scheme serves as a middle-ground between the SEQ and TBT configurations and could be used by *@HOME* type applications.

Note that each scheme aims to optimize a different subset of the four design parameters. In Tables 3 and 4, we summarize each configuration's pros (with positive signs) and cons (with negative signs). IOR represents the IO resolution.

Scheme	*Seek Overhead*	*IOR*	*Storage*
Sequential	++	– –	*NA*
TBT	++	++	*NA*
PTBT	++	+	*NA*

Table 3: Scheme Summary for Read operations.

Scheme	*Seek Overhead*	*IOR*	*Storage*
Sequential	++	*NA*	++
TBT	– –	*NA*	– –
PTBT	–	*NA*	–

Table 4: Scheme Summary for Write operations.

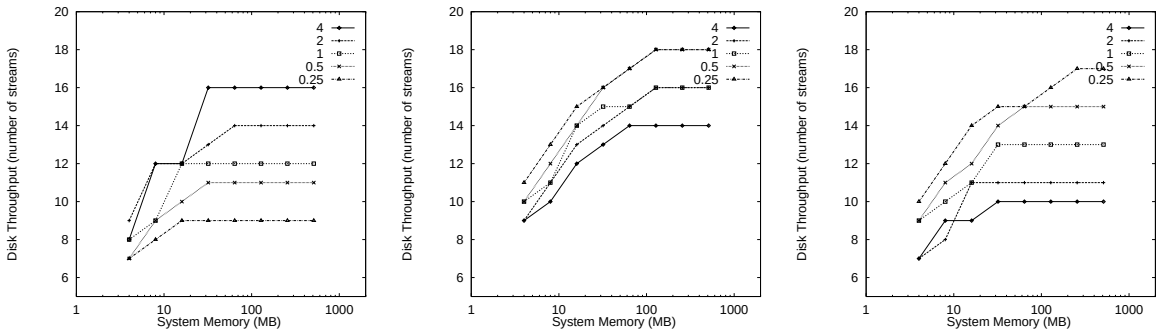
6.1.1 Available System Memory

In this section, we compare the memory requirement for each of the three sample configurations of the adaptive tree data organization scheme using disk traces. These disk traces were generated so as to mimic the IO load of a media server. The trace executions were performed using the data placement and IO scheduling strategies of the IMP system.

Assuming a given amount of main memory, first we calculated the maximum IO cycle time expendable to support N users. Next, we obtained the actual IO cycle time from disk runs. If the disk runs were shorter than the analytically computed IO time cycle, we conclude that the disk can support N users. If not, we repeat the above experiment with $(N-1)$ users. We continue this iterative process till we can obtain the correct value of N. For this value of N, we can be sure that the user streams will be hiccup-free. We repeat the same experiment by assuming different memory sizes. For the following traces conducted using the Scsibench tool, we assumed that the fraction of user requests that are for fast-scan streams is 0.2. We evaluate the effect of the varying this parameter in greater detail subsequently. We assume that the peak data consumption and input rates are 6.4 Mbps (the Standard DTV broadcast rate). Further, we assume that the frame-rate for fast-scans is 5 fps (Please refer to Section 3 for an explanation of why we choose a lower frame-rate for fast-scan streams). We use the same numbers for all subsequent experiments unless specified explicitly.

Figure 5(a), 5(b), and 5(c) present the disk throughput for the SEQ, PTBT, and TBT configurations respectively against varying memory size (M) and for different values of the write-read ratio ($\rho = 0.25, 0.5, 1, 2, 4$) [4]. We note that for the SEQ scheme, we need as little

[4]We were restricted to these numbers because the maximum transfer rate of the disk that we used to perform traces was 19 MBps. This restricted the number of MPEG2 streams that the disk bandwidth can support to less



(a) SEQ (b) PTBT (c) TBT

Figure 5: Disk throughput (number of streams) for Adaptive Tree configurations.

as 32 MBytes of memory in order to maximize disk throughput. For the PTBT and TBT schemes the corresponding numbers are 128 MBytes and 64 MBytes respectively. Looking at these figures from another perspective, a mere 32 MBytes of main memory is sufficient to drive the disk throughput to almost 90% of the maximum achievable value. The is the case because for this amount of memory, the system can buffer enough data so that all disk accesses are only in large chunks. When the disk is accessed in large chunks, disk latency is much lesser in comparison to the time spent in data transfer. At this point, the bottleneck is the raw data transfer rate of the hard drive, which can only support a fixed maximum number of high bandwidth streams.

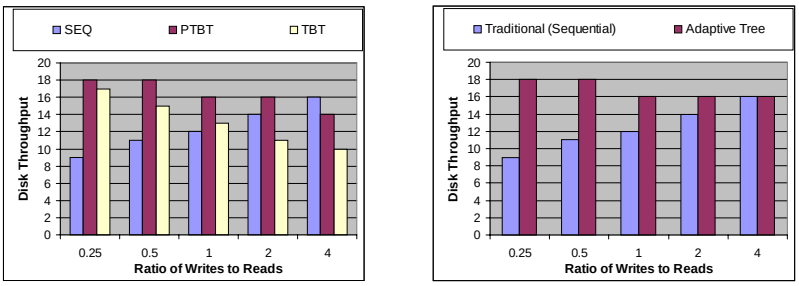
6.1.2 Write-Read Ratio

In this section, find out the variation in disk throughput under different write-read ratios using disk traces. We also compare the IMP system against the traditional UNIX-like file-system that tries to store file data sequentially on disk and present the improvement in performance. Finally, we present analytical results for wider ranges of the write-read ratio.

Figure 6(a) compares the relative performance of IMP system in each of the sample configurations [SEQ, PTBT, and TBT]. Since the SEQ scheme is optimized for large number of writes, it achieves a high throughput for write intensive load. PTBT performs optimally in mid-ranges and TBT performs well for read intensive load. For different values of ρ , different configurations achieve the highest throughput thus defining a distinct *ideal* region.

Next, we compare performance improvement of the IMP system over the traditional approach to storing and retrieving data. A traditional file system like Unix is optimized for sequential access. However, it is not optimized to handle high bandwidth and high volume

than 20. Hence experimentation beyond these ranges was meaningless. However, we do present results for other values through analytical estimation later.



(a) (b)

Figure 6: Throughput Comparison. (a) Different configurations of the Adaptive Tree Scheme. (b) Improvement of the Adaptive Tree Scheme over Traditional sequential placement of data.

data. Moreover, for an interactive video application, access to data is not always sequential. For instance, a fast-scan stream only needs to access key frames from the entire file. In addition, traditional file systems do not perform bit-rate matching (Section 3.2) to optimize disk performance. The IMP system provides "fine-grained" device management strategies for supporting interactive high bandwidth streaming data formats. In addition, it can adapt itself to changing workload and can support data with multiple bit-rates. Figure 6(b) shows that for different workload configurations, IMP either outperforms or equals the performance of a traditional file-system. Throughput gains can be as much as 100% using IMP.

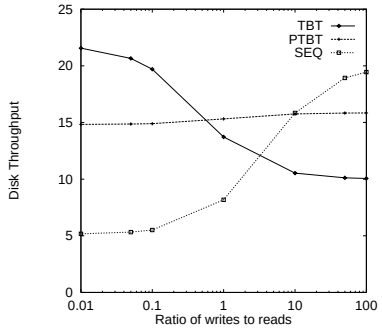
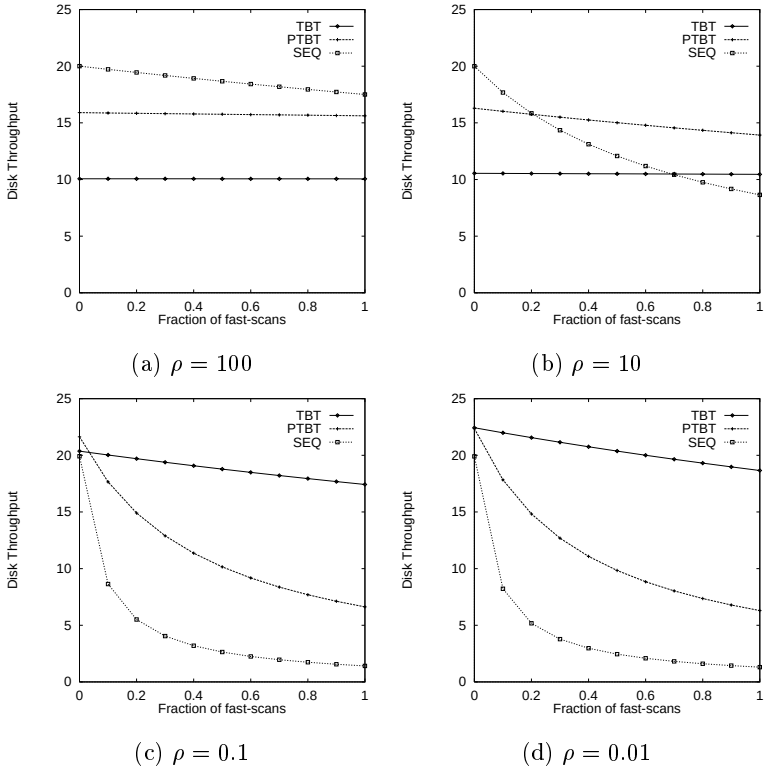


Figure 7: Throughput for the three sample configurations.

To circumvent the physical limitations of the hard drive, we present the throughput evaluation of the three sample configurations by analytical estimation. We use parameters for the Seagate ST39102LW disk, presented in Table 2, and complete seek curves obtained using our disk profiler, to perform our analysis. Note that although a different set of disk parameters can lead to different absolute performance values, the relative performance numbers remain the same. In Figure 7, the x-axis represents the write-read ratio, and the y-axis shows the disk throughput achievable by the configurations. We can see clearly that for different values of ρ, different configurations achieve the highest throughput,thus defining a distinct *ideal* re-

gion. This means that the density of the adaptive tree has to be configured intelligently and dynamically for optimal performance.

6.1.3 Fraction of Interactive Streams



(a) $\rho = 100$ (b) $\rho = 10$

(c) $\rho = 0.1$ (d) $\rho = 0.01$

Figure 8: Disk Throughput vs. Fast-scan Percentage.

In this section, we evaluate the effect of varying the fraction of interactive streams (f). To this end, we plot the throughput numbers for each scheme separately for varying percentages of fast-scan streams. Figure 8 verifies the following:

1. *High* ρ ($\rho = 100$): For write intensive load, we find that the SEQ configuration performs much better than its denser counterparts. However, we also observe that as the percentage of fast-scan requests increases, the SEQ configuration suffers from poor IO resolution. In this region, the TBT and PTBT configurations are almost unaffected by the percentage of fast-scans since they have very good IO resolution for fast-scan streams, which in this case are also less in number.

2. *Medium* ρ ($\rho = 0.1$ *to* 10): For medium values of ρ, the best placement configuration varies greatly with the percentage of fast-scan requests. For $\rho = 10$, if the percentage of fast-scans is small, SEQ outperforms the TBT and PTBT configurations. If the number of fast-scans rises above 60%, the SEQ configuration starts performing worse due to its poor IO resolution

for fast-scans. For $\rho = 0.1$, however, SEQ starts to degrade at a much lower percentage of fast-scans since the read streams are now in the majority. When reads dominate, IO resolution becomes critical. Therefore, the TBT scheme, which has perfect IO resolution at all speeds, outperforms the other schemes.

3. *Low* ρ ($\rho = 0.01$): For read intensive load (e.g., $\rho = 0.01$), there are many more requests for read than for write streams. A marked difference in this operating region is that the performance of the TBT and PTBT configurations now starts degrading with increasing percentage of fast-scans. This means that the higher data-rate requirement for fast-scans starts playing a significant role in deciding the throughput as the fast-scan streams begin to dominate. PTBT now performs worse than the TBT configuration due to poorer IO resolution. This is the operating region for the denser configurations.

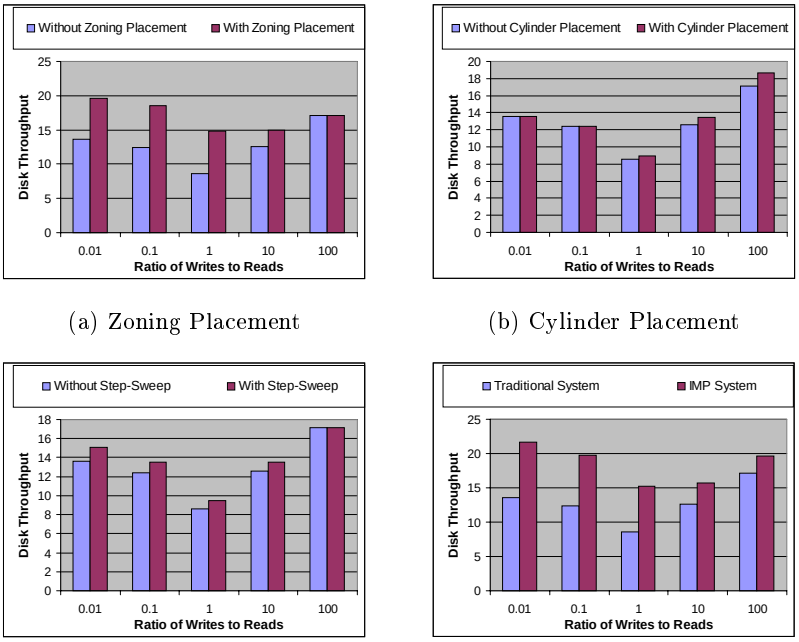
6.2 Data Management Strategies

Next, we compare the IMP system against an alternate system that uses the traditional *sweep* disk scheduling algorithm. Using sweep, the disk arm moves from the outermost cylinder to the innermost, servicing requests along the way. In traditional systems, the sweep algorithm is known to perform well for a wide range of applications. Most modern operating systems use sweep algorithm for disk IO scheduling. In order to make a fair comparison, we give the alternate system the benefit of using the adaptive tree data organization scheme to achieve good IO resolution. In addition, for each of the following evaluations, we assume that the system is dynamically tuned to the "ideal" adaptive tree configuration under changing write-read ratios. In essence we try to capture the effect of fine-grained device management of the IMP system in the form of intelligent *zoning*, *cylinder placement* and *step-sweep* scheduling on throughput improvement.

We start with exploring the individual contributions of each of our fine-grained strategies in improving throughput of the system. We compare against a traditional UNIX-like file-system that uses *sweep* scheduling. Then, we add our device management strategies, one at a time, to examine the marginal improvement in throughput due to each strategy. Finally, we present the cumulative effect of these disk management strategies, in increasing the throughput of the system.

6.2.1 Zoning placement

Figure 9(a) compares the throughput of the baseline sweep with sweep using zoning placement. Zoning placement improves throughput for read intensive loads by as much as 40%. For high write/read ratios, our adaptive tree scheme adopts the ideal sequential (SEQ) configuration and hence zoning has no effect.



(a) Zoning Placement

(b) Cylinder Placement

(c) Step-sweep Scheduling

(d) Total Improvement

Figure 9: Throughput Improvement due to Device Management strategies.

6.2.2 Cylinder placement

Figure 9(b) compares the the throughput of the baseline sweep with and without cylinder placement. Cylinder placement improves write performance by reducing the seek overhead for write operations. It increases the throughput by about 10% for write intensive load.

6.2.3 Step-sweep IO scheduling

Next, we examine the step-sweep IO scheduling algorithm. Step-sweep increases throughput by decreasing memory use. It reduces memory use by minimizing IO variability and seek overhead. Figure 9(c) shows that without step-sweep scheduling, there is a 5 to 10% degradation in throughput.

Finally, Figure 9(d) shows the cumulative effect of our fine-grained device management strategies. The IMP system offers a performance gain of as much as 75% over traditional systems. This highlights the importance of performing fine-grained storage management using the IMP system.

As regards response time, a detailed evaluation is beyond the scope of this paper and left to future work. However, it is clear that in case of step-sweep, which serves new requests immediately in a reservation time-slot, the worst-case response time is bounded by the time required to complete the current non-preemptable IO. This time is typically of the order of

a few milliseconds. Traditional IO schedulers like sweep as well as GSS [27], have worst-case response times of the order of minutes under heavy load, and at least of the order of seconds under average load conditions.

6.3 Observations

We conclude our evaluation section by making the following key observations:

- High-level data organization in the form of the adaptive tree scheme is crucial to maintain disk throughput. The effect of the three system configuration parameters of memory, write-read ratio, and fraction of interactive streams on the performance of the IMP system are summarized as follows:

  1. Increasing the system memory beyond a certain threshold does not significantly increase system throughput. At this point, the disk throughput becomes the bottleneck. Beyond this threshold, the disk throughput can only be increased by increasing IO efficiency by using the adaptive tree scheme for data organization.

  2. Using the adaptive tree MPEG data organization scheme, the IMP system can optimize for read intensive as well as write intensive loads. The achieved throughput gains are as much as 100% more than that of a traditional file-system for a large range of write-read ratios.

  3. Each sample configuration [SEQ,PTBT,and TBT], of the adaptive tree scheme operate efficiently for a unique sub-configuration defined by the values of write-read ratio (ρ) and interactive stream fraction (f). To maximize system throughput, choosing the right configuration at all times is crucial.

- We evaluated the performance gain due each of our data management strategies, namely, zoning placement, cylinder placement and step-sweep IO scheduling. We summarize our findings as follows:

  1. Zoning placement matches stream bit-rates to disk zone transfer rates so as to maximize data throughput of the disk for serving continuous media streams. It improves throughput by as much as 40% for read intensive loads and by less than 20% for write intensive loads.

  2. Cylinder placement improves write performance by reducing the seek overhead for write operations. It increases the throughput by about 10% for write intensive load.

  3. Step-sweep increases throughput by decreasing memory use. Throughput gains due to step-sweep are about 5 to 10%.

  The cumulative effect of the above strategies make it possible for the IMP system to offer performance gains of as much as 75% over traditional systems.

7 Related Work

Video broadcasting and multicasting has been used in some cable-based or satellite-based movie-on-demand channels, in which requests for a movie arriving within a period of time are "grouped" (i.e., batched) together and served with a single stream [14, 13, 23]. Schemes have been proposed to support viewing interactivity in a broadcast (or multicast) environment [1, 6, 16]. However, these schemes often double the bandwidth requirement for clients or require large client buffering. In this paper, we propose a client/server dual architecture that manages media data between broadcasters/multicasters and clients for enabling interactivity.

Several schemes have been proposed for supporting fast-scans. These schemes can be divided into three approaches:

1. *Increase playback rate.* This approach increases the playback frame rate for supporting a fast-scan operation. Suppose the regular playback rate is 30 frames a second. This method plays back 60 frames for a two-times speedup fast forward. This method is impractical since no TV can display more than 30 frames per second. Second, the IO bandwidth, memory use, and CPU overhead can be exceedingly high.

2. *Use separate fast-scan streams [5, 20, 21, 22].* This approach cannot be used in the broadcast (or multicast) scenario since a program is broadcast at a single rate. We can add a unicast channel to deliver fast-scan data, but this requires additional network bandwidth. The study of [4] proposes a client-side approach that re-encodes frames during normal playback and saves them for replay. This approach can be CPU-intensive since most compression schemes are encoding-side heavy and hence may hinder a CPU from decoding more streams.

3. *Skip frames in the regular stream [8].* The frame-skipping approach can cause low IO-resolution and consequently low system throughput if not designed carefully. Suppose a video is laid out sequentially on disk, and the scheme attempts to retrieve every third frame. If we break a sequential IO into small IOs to read every third frame, we can end up with worse, rather than better, IO performance. We believe that skipping frames is nevertheless the right approach for supporting fast-scans under our dual client/server setting. In this study, we propose remedies to improve IO resolution and to reduce IO overhead to implement this approach efficiently.

Data placement and IO scheduling for improving disk throughput has been studied extensively in the context of multimedia file systems [3, 15, 20, 21, 22, 27]. More work has been done than can be cited here. However, simultaneous storage and retrieval of video streams has not been explored in any of the earlier studies. Most work has been done on read-only systems like video-on-demand, where videos are assumed to be stored on the disk to start with. Even in read-only systems, low-level device optimizations have not been explored. Our work offers a low-level optimized solution to simultaneous storage and retrieval of continuous media. Our low-level data placement and IO scheduling strategies can also improve the performance

of read-only systems like a web-based multimedia server. Specifically, we can differentiate our approach from previous efforts in the following two aspects:

1. *Fine-grained* device management: We collect detailed disk parameters directly from a disk to make more effective real-time device management decisions.

2. *Integrated* device management: From disk feature extraction, data organization, and data placement, to IO scheduling, we provide an integrated strategy. We show that tradeoffs often exist between design parameters and propose scalable methods to find optimal tradeoffs under different workload scenarios.

Disk profiling has been addressed before [26]. However, we believe that our study is more extensive and can be applied to both SCSI as well as IDE disks. Moreover, at the time of writing this paper, the previous profiling tool was not publicly available. The profiling tool that we built is open source [7] and can be easily modified to suit user requirements.

8 Conclusion

In this paper we have proposed a novel client/server dual architecture and design of IMP, an interactive media proxy that can enable a non-interactive broadcast stream to become an interactive one. Such an architecture is both cost-effective and more practical than the traditional server-based model.

We studied the key design parameters for building the IMP system and analyzed the tradeoffs between these parameters. To maximize system throughput, we proposed and evaluated fine-grained device management strategies. After extensive analysis and experimentation of the IMP system, we conclude our study as follows: High-level data organization in the form of an *adaptive tree* structure is crucial to maintain high throughput under changing write-read request ratio. The data placement strategies of *zoning* and *cylinder placement* employed in IMP improve the throughput of the system significantly. *Step-sweep* IO scheduling further improves the throughput by minimizing seek overhead and IO variability. IMP offers an overall performance gain of as much as 75% over traditional systems, thus underlining the importance of performing fine-grained device management for interactive media services.

References

[1] K. C. Almeroth and M. H. Ammar. The use of multicast delivery to provide a scalable and interactive video-on-demand service. *IEEE Journal on Selected Areas in Communications*, 14(6):1110–1122, August 1996.

[2] S.-H. G. Chan and F. Tobagi. Distributed servers architecture for networked video services. *IEEE/ACM Transactions on Networking*, to appear.

[3] E. Chang and H. Garcia-Molina. Effective memory use in a media server. *Proceedings of the 23rd VLDB Conference*, pages 496–505, August 1997.

[4] M. S. Chen and D. D. Kandlur. Downloading and stream conversion: Supporting interactive playout of videos in a client station. *ICMCS*, May 1995.

[5] M. S. Chen and D. D. Kandlur. Stream conversion to support interactive video playout. *IEEE Multimedia*, 3(2):51–58, Summer 1996.

[6] A. Dan, P. Shahabuddin, D. Sitaram, and D. Towsley. Channel allocation under batching and VCR control in video-on-demand systems. *Journal of Parallel and Distributed Computing*, 30(2):168–179, 1995.

[7] Z. Dimitrijevic, D. Watson, and A. Acharya. Scsibench. *http://www.cs.ucsb.edu/∼zoran/scsibench/*, 2000.

[8] W. Feng, F. Jahanian, and S. Sechrest. Providing vcr functionality in a constant quality video-on-demand transportation service. *ICMCS*, June 1996.

[9] K. Hua and S. Sheu. Skyscraper broadcasting: A new broadcasting scheme for metropolitan video-on-demand systems. *ACM Sigcomm*, pages 89–101, Sept. 1997.

[10] A. N. S. I. Scsi-2 specification x3t9.2/375r revision 10l, January 1995.

[11] T. Johnson and A. Zhang. A framework for supporting quality-based presentation of continuous multimedia streams. *Proceedings of the 4th IEEE Conference on Multimedia Computing and Systems*, pages 169–176, June 1996.

[12] M. G. Kienzle, A. Dan, and D. Sitaram. Using tertiary storage in video-on-demand servers. *Proceedings of IEEE CompCon*, 1995.

[13] V. O. K. Li and W. Liao. Distributed multimedia systems. *Proceedings of the IEEE*, 85(7):1063–1108, July 1997.

[14] T. Little and D. Venkatesh. Prospects for interactive video-on-demand. *IEEE Multimedia Magazine*, pages 14–24, Fall 1994.

[15] R. Ng and J. Yang. Maximizing buffer and disk utilizations for news on-demand. *Proceedings of the 20th VLDB Conference*, pages 451–462, 1994.

[16] W.-F. Poon and K.-T. Lo. Design of multicast delivery for providing VCR functionality in interactive video-on-demand systems. *IEEE Transactions on Broadcasting*, 45(1):141–8, March 1999.

[17] R. Rangaswami, E. Chang, and Z. Dimitrijevic. Disk-aware data mangement for interactive media services (extended version). *http://www-db.stanford.edu/∼echang/ics-extended.ps, Technical Report*, March 2001.

[18] M. Rosenblum and J. Ousterhout. The design and implementation of a log-structured file system. 1991.

[19] C. Ruemmler and J. Wilkes. An introduction to disk drive modeling. *Computer*, 2:17–28, 1994.

[20] P. Shenoy and H. Vin. Efficient support for interactive operations in multi-resolution video servers. *ACM Multimedia Systems*, 7(3), May 1999.

[21] P. J. Shenoy and H. M. Vin. Efficient support for scan operations in video servers. *Proceedings of the 3rd ACM International Conference on Multimedia*, pages 131–140, 1995.

[22] W. Tavanapong, K. Hua, and J. Wang. A framework for supporting previewing and vcr operations in a low bandwidth environment. *Proceedings of the 5th ACM Multimedia Conference*, November 1997.

[23] F. A. Tobagi. Distance learning with digital video. *IEEE Multimedia Magazine*, pages 90–94, Spring 1995.

[24] C. Wang, D. J. Ecklund, E. F. Ecklund, V. Goebel, and T. Plagemann. Design and implementation of a lod system for multimedia supported learning for medical students. *iWorld Conference on Educational Multimedia, Hypermedia & Telecommunications ED-MEDIA*, June 2001.

[25] J. Wolf, , P. Yu, and H. Shachnai. Disk load balancing for video-on-demand systems. *ACM Multimedia Systems*, December 1997.

[26] B. Worthington, G. Ganger, Y. Patt, and J. Wilkes. Online extraction of scsi disk drive parameters. *Proc. of ACM Sigmetrics Conference*, pages 146–156, 1995.

[27] P. Yu, M.-S. Chen, and D. Kandlur. Grouped sweeping scheduling for DASD-based multimedia storage management. *Multimedia Systems*, 1(1):99–109, January 1993.